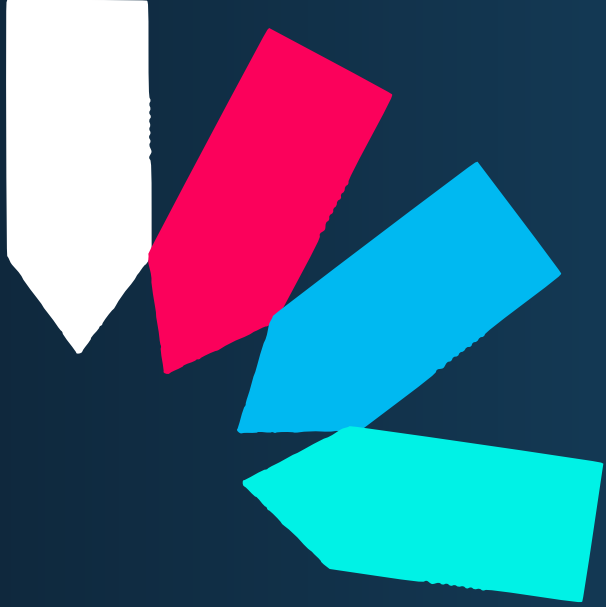
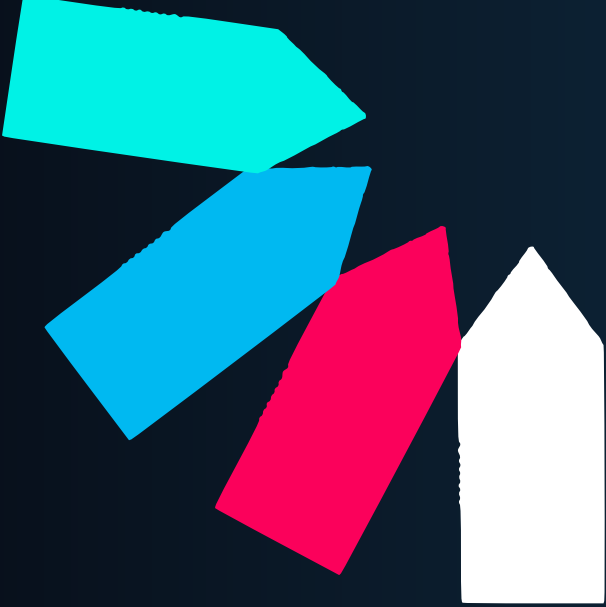




Anatomy of a JWT attack



By Chaos



 **warning**

**This document is only for
educational purposes.**

**The author will approve of
no abuse.**



Contents

3	 آناتومی هک JWT
3	 JWT چیست و چرا مهم است؟
3	 عدم تأیید امضا (Signature Not Verified)
5	 کلیدهای HMAC ضعیف (Brute Force HS256)
5	 حمله سردرگمی الگوریتم (RS256 → HS256)
6	 حمله سردرگمی الگوریتم (ES256 → HS256)
6	 تزریق kid (Key ID Manipulation)
7	 حمله JWK جاسازی شده (CVE-2018-0114)
8	 سوء استفاده از هدرهای JKU / X5U
8	 سردرگمی ادعاها (Missing aud, iss, sub Checks)



آناتومی هک JWT

JWT چیست و چرا مهم است؟

JWT (JSON Web Token) یک روش فشرده برای نمایش ادعاها (claims) بین دو طرف است. می‌توانید آن را به عنوان یک بسته JSON امضا شده در نظر بگیرید که در برنامه‌های وب برای مدیریت session ها، مجوزها و هویت کاربر بدون نیاز به ذخیره state در سمت سرور استفاده می‌شود.

ساختار JWT

<base64url(Header)>.<base64url(Payload)>.<base64url(Signature)>

- **هدر (Header)** الگوریتم امضا (alg) و نوع توکن (typ) را مشخص می‌کند.
- **payload (Payload)** شامل ادعاها (claims) مانند sub ، admin ، exp ،iat و غیره است.
- **امضا (Signature)** تضمین می‌کند که داده‌ها دستکاری نشده‌اند.

مثال payload

```
{
  "sub": "user1",
  "admin": false
}
```

اطلاعات بیشتر درباره JWT

- <https://jwt.io/introduction>
- <https://auth0.com/docs/secure/tokens/json-web-tokens>

عدم تأیید امضا (Signature Not Verified)

این کشنده‌ترین و رایج‌ترین اشتباه است. برخی توسعه دهندگان از `decode()` برای تجزیه JWTs بدون تأیید امضا با `verify()` استفاده می‌کنند. اگر backend تأیید امضا را رد کند، anyone می‌تواند (any field مثلاً `admin: true`) را تغییر دهد، یا حتی امضا را به طور کامل حذف یا تغییر دهد، و سرور همچنان توکن را می‌پذیرد. این امنیت و یکپارچی فرآیند احراز هویت را به خطر می‌اندازد.



مثال عملی:

- توکن اصلی:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1c2VyMSIsImFkbWluljpmYWxzZX0.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

- توکن دستکاری شده (بدون تأیید امضا):

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1c2VyMSIsImFkbWluljpmYWxzZX0.XXX
```

حمله alg: none

JWTs الگوریتم امضا را در هدر مشخص می کنند، و alg: none به معنای عدم وجود امضا است. که در اصل برای دیباگینگ در نظر گرفته شده بود، اما برخی کتابخانه ها (یا پیکربندی های careless) همچنان آن را allow می کنند.

مراحل حمله:

1. تغییر "alg" در هدر به "none"
2. حذف بخش امضا
3. تغییر payload به دلخواه
4. reassemble کردن header.payload بدون امضا
5. ارسال توکن (اگر accepted شود، برنامه به داده های unsigned اعتماد می کند. بازی تمام است.)

مثال عملی:

- توکن اصلی:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1c2VyMSIsImFkbWluljpmYWxzZX0.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

- توکن حمله alg: none

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1c2VyMSIsImFkbWluljpmYWxzZX0.XXX
```

راهکارهای پیشگیری:

- همیشه از verify() به جای decode() استفاده کنید.
- الگوریتم none را در کتابخانه JWT غیرفعال کنید.



- از کلیدهای قوی و مدیریت امن کلیدها استفاده کنید.
- به طور منظم کتابخانه های JWT را به روز کنید.

کلیدهای HMAC ضعیف (Brute Force HS256)

وقتی برنامه ها از الگوریتم **HS256** استفاده می کنند، همان کلید برای امضا و تأیید توکن استفاده می شود. اگر این کلید ضعیف باشد (مثلاً secret، 123456، نام برنامه و...)، می توان آن را به صورت آفلاین brute-force کرد.

مراحل حمله:

۱. گرفتن یک JWT معتبر
۲. استفاده از ابزارهایی مانند **hashcat** یا **jwt-tool** برای brute-force کردن کلید:

```
bash  
  
hashcat -a 0 -m 16500 <jwt> <wordlist>
```

۳. جعل توکن با payload دلخواه

نکته: این حمله به خصوص در محیط های dev/staging، پروژه های متن باز و تنظیمات عجله ای به خوبی جواب می دهد.

حمله سردرگمی الگوریتم (RS256 → HS256)

فرض کنید برنامه از **RS256** استفاده می کند که بر اساس جفت کلید خصوصی/عمومی کار می کند:

- سرور با کلید خصوصی امضا می زند
- با کلید عمومی تأیید می کند

اما اگر برنامه به فیلد alg از JWT اعتماد کند، می توانید:

- RS256 را به HS256 تغییر دهید
- از کلید عمومی (که ممکن است در دسترس باشید) به عنوان کلید HMAC استفاده کنید
- یک توکن با payload دلخواه خود امضا کنید

نکته: سرور فکر می کند در حال تأیید با RSA است، اما در واقع یک HMAC جعلی را تأیید می کند. دسترسی اعطا می شود!



حمله سر در گمی الگوریتم (ES256 → HS256)

ایده مشابه حمله قبل با الگوریتم متفاوت. به جای RSA، سرور از **ECDSA (ES256)** استفاده می کند.

اگر برنامه الگوریتم مورد انتظار را enforce نکند:

- { alg: ES256 } را به { alg: HS256 } تغییر دهید
- از کلید عمومی ECDSA به عنوان کلید HMAC استفاده کنید
- payload خود را امضا کنید

تزریق kid (Key ID Manipulation)

JWTs می توانند شامل یک فیلد kid در هدر باشند که مشخص می کند از کدام کلید برای تأیید توکن باید استفاده شود.

اما اگر برنامه کلیدها را از سیستم فایل با استفاده از kid بارگذاری کند، یا یک query خام بانکی با kid انجام دهد، می توانید کارهایی مانند این انجام دهید:

- سرور یک کلید خالی می خواند → kid: ../../../../dev/null
- SQL تزریق → kid: ' UNION SELECT 'fake-key' --

نکته: اگر درست استفاده شود، می توانید برنامه را به کلیدی که شما کنترل می کنید اشاره دهید، یا آن را مجبور به استفاده از یک کلید خالی کنید و توکن های خود را امضا کنید.

مثال عملی حمله kid

- هدر اصلی:

```
{
  "alg": "RS256",
  "kid": "123"
}
```

- هدر دستکاری شده:



```
{
  "alg": "RS256",
  "kid": "../../../dev/null"
}
```

یا برای تزریق SQL

```
{
  "alg": "RS256",
  "kid": "' UNION SELECT 'fake-key' --"
}
```

راهکارهای پیشگیری:

- همیشه الگوریتم مورد انتظار را enforce کنید
- از کلیدهای قوی و تصادفی برای HMAC استفاده کنید
- فیلد kid را اعتبارسنجی کنید
- از دسترسی‌های محدود برای خواندن کلیدها استفاده کنید
- به طور منظم کتابخانه‌های JWT را به روز کنید

حمله JWK جاسازی شده (CVE-2018-0114)

JWTs از یک فیلد اختیاری jwk پشتیبانی می‌کنند که کلید عمومی را مستقیماً در هدر توکن جاسازی می‌کند. اگر سرور هر کلیدی از این فیلد را بدون اعتبارسنجی بپذیرد، آسیب‌پذیر است.

مراحل حمله:

- یک جفت کلید RSA خود را generate کنید
- JWT را با کلید خصوصی خود امضا کنید
- کلید عمومی را در هدر jwk جاسازی کنید و آن را ارسال کنید

نکته: برنامه از کلید جاسازی شده شما برای تأیید توکن جعلی شما استفاده می‌کند. دور زدن کامل!



سوء استفاده از هدرهای X5U / JKU

فیلدهای jku و x5u به JWT اجازه می‌دهند به URL های خارجی برای کلیدها یا گواهی‌ها reference دهد. اگر backend این کلیدها را بدون تأیید منبع آن‌ها fetch کند، آسیب‌پذیر است.

مراحل حمله:

۱. مهاجم یک JWKS یا گواهی X.509 مخرب در URL خود host می‌کند.
 ۲. یک توکن جعلی با کلید خصوصی خود امضا می‌کند و jku/x5u را روی URL خود تنظیم می‌کند.
 ۳. سرور کلید مهاجم را fetch می‌کند، امضا را تأیید می‌کند و توکن را معتبر می‌پذیرد.
- نکته:** این نه تنها تزریق کلید است، بلکه می‌تواند یک بردار برای SSRF نیز باشد.

سردرگمی ادعاها (Missing aud, iss, sub Checks)

اگر برنامه claims مانند audience یا issuer را validate نکند، مهاجمان می‌توانند:

- از توکن‌های صادر شده برای یک سرویس دیگر استفاده کنند
- توکن‌ها را در میکروسرویس‌ها یا API ها reuse کنند
- به صورت افقی امتیازات را افزایش دهند

نکته: این مورد به طور تعجب‌آوری در معماری‌های مدرن و توزیع شده رایج است.

توکن‌های بدون انقضا / با عمر طولانی

JWTs باید به سرعت منقضی شوند. اگر یک توکن claim exp نداشته باشد، یا exp برای سال‌های آینده تنظیم شده باشد، می‌تواند به طور نامحدود reuse شود و به مهاجمان اجازه دسترسی مداوم بدهد.

نکته: اغلب با سایر تکنیک‌ها برای حفظ دسترسی پس از compromise جفت می‌شود.



مثال عملی حمله JWK جاسازی شده:

- هدر توکن دستکاری شده:

```
{
  "alg": "RS256",
  "jwk": {
    "kty": "RSA",
    "n": "مقدار کلید عمومی مهاجم",
    "e": "AQAB"
  }
}
```

مثال عملی حمله JKU

- هدر توکن دستکاری شده:

```
{
  "alg": "RS256",
  "jku": "https://attacker.com/malicious_keys.json"
}
```

راهکارهای پیشگیری:

- همیشه منبع کلیدهای خارجی را اعتبارسنجی کنید
- فیلدهای aud، iss و sub را validate کنید
- زمان انقضای کوتاه برای توکن‌ها را کوتاه کنید
- از allowlist برای URL های قابل اعتماد استفاده کنید
- به طور منظم توکن‌ها را بررسی و expire کنید

